

AM6513: ACFD Assignment 1.2

Problem 3: Multi-Block with Coordinate Mapping

Technical Documentation

Contents

1	Introduction	2
2	Workflow	2
3	pipebend.cpp	2
3.1	User Inputs	2
3.2	Computational Domain	3
3.3	Coordinate Transformations	3
3.4	Mesh Conformity Check	4
3.5	Multi-Block Assembly	4
3.6	Mesh Quality Check and Export	4
4	MeshConformity.cpp	5
4.1	checkConformity: Public Interface	5
4.2	checkInterface: Interface Verification	6
5	MeshQuality.cpp	6
5.1	Mathematical Diagnostics	7
5.1.1	Signed Area	7
5.1.2	Scaled Corner Jacobian	7
5.1.3	Aspect Ratio	7
5.2	Core Diagnostic Implementations	7
5.3	Quality Evaluation Logic	8
6	Automated Testing Suite	9
6.1	Geometry Initialization	9
6.2	Computational Domain Discretization	10
6.3	Coordinate Mappings	10
6.4	Automated Validation	11
7	Summary	11

1 Introduction

This document describes the implementation developed for Problem 3 using the *Pipe Bend* case as the demonstration problem. The case consists of three computational blocks: an inlet section, a pipe-bend section, and an outlet section.

The implementation extends the existing `aquaMesh` repository with the following components:

- **Coordinate mapping methods:** `PipeBend`, `inlet`, and `outlet`, implemented within the existing `CoordinateMapping` class.
- **MeshConformity:** checks conformity between adjacent mesh blocks.
- **MeshQuality:** evaluates the quality of transformed mesh cells.
- **Conformity tests:** validates the added mesh-conformity and quality functionality.

The executable `pipebend.cpp` combines these components to generate, validate, and export the final multi-block mesh.

2 Workflow

The main workflow implemented in `pipebend.cpp` is:

1. Create three rectangular surfaces using points, lines, and surface objects.
2. Generate a structured Cartesian (ξ, η) mesh for each surface.
3. Apply coordinate mappings to transform the computational blocks into the inlet, pipe bend, and outlet geometries.
4. Check conformity between adjacent transformed meshes.
5. Assign block identifiers to the individual meshes.
6. Assemble the blocks into a `MultiBlockMesh`.
7. Evaluate mesh quality and reject inverted or low-quality cells.
8. Export the final mesh as a `.vtk` file only when all checks pass.

3 pipebend.cpp

3.1 User Inputs

The following parameters control the geometry and discretization:

```

1 double angle = acos(-1.0);
2 double rinner = 5.0; // Pipe inner radius
3 double router = 10.0; // Pipe outer radius
4 double lI = 5.0; // Inlet pipe length
5 double lO = 5.0; // Outlet pipe length
6 int Nr = 20; // Radial nodes
7 int Na = 15; // Axial nodes

```

```
8  int N1 = 10;           // Lateral nodes for pipes
```

3.2 Computational Domain

Three ξ - η computational meshes are generated from the previously constructed rectangular surfaces.

```
1  Mesh xi_eta_1, xi_eta_2, xi_eta_3;
2
3  xi_eta_1.generateCartesian(
4      block1Surface,
5      Nr,
6      Na
7  );
8
9  xi_eta_2.generateCartesian(
10     block2Surface,
11     Na,
12     N1
13 );
14
15 xi_eta_3.generateCartesian(
16     block2Surface,
17     N1,
18     Na
19 );
```

3.3 Coordinate Transformations

The first computational block is mapped into the pipe bend. The remaining blocks are mapped into the inlet and outlet sections.

```
1  Mesh pipemesh = xi_eta_1;
2
3  // Transform the first block into the pipe bend
4  CoordinateMapping::PipeBend(
5      pipemesh,
6      angle,
7      rinner,
8      router
9  );
10
11 // Transform the remaining blocks into inlet and outlet sections
12 Mesh inletmesh = xi_eta_2;
13 Mesh outletmesh = xi_eta_3;
14
15 CoordinateMapping::inlet(
16     inletmesh,
17     lI,
18     rinner,
19     router
20 );
21
```

```

22 CoordinateMapping::outlet(
23     outletmesh,
24     l0,
25     angle,
26     rinner,
27     router
28 );

```

3.4 Mesh Conformity Check

The transformations are rejected if the interfaces between adjacent meshes are not conformal.

```

1  // Check whether the transformed blocks are conformal
2  if (
3      !MeshConformity::checkConformity(inletmesh, pipemesh) ||
4      !MeshConformity::checkConformity(pipemesh, outletmesh)
5  ) {
6      cout << "Meshes are not conformal. Transformations are rejected\n";
7      return -1;
8  }

```

3.5 Multi-Block Assembly

The individual blocks are assembled into a single multi-block mesh.

```

1  MultiBlockMesh mesh;
2
3  mesh.addBlock(block1);
4  mesh.addBlock(block2);
5  mesh.addBlock(block3);
6
7  mesh.generateFromMesh();

```

3.6 Mesh Quality Check and Export

The generated mesh is evaluated using the MeshQuality module. Export is performed only when no inverted or low-quality cells are detected.

```

1  // Evaluate mesh quality
2  auto results = MeshQuality::evaluate(mesh, 5.0);
3  MeshQuality::report(results);
4
5  // Check quality constraints
6  bool hasInverted =
7      MeshQuality::hasInvertedCells(results);
8
9  bool hasLowQuality =
10     MeshQuality::hasLowQualityCells(results);
11
12  if (hasInverted || hasLowQuality) {
13
14     if (hasInverted && !hasLowQuality) {
15         cout << "Transformation produces inverted cells.\n";

```

```

16     }
17     else if (hasLowQuality && !hasInverted) {
18         cout << "Transformation produces low quality cells.\n";
19     }
20     else {
21         cout << "Transformation produces both inverted "
22             "and low quality mesh.\n";
23     }
24
25     cout << "Mesh generation rejected\n";
26     return -1;
27 }
28 }
29 else {
30
31     MeshWriter::writeVTK(
32         mesh,
33         "pipebendMultiBlock.vtk"
34     );
35
36     cout << "Mesh generation successful";
37 }

```

4 MeshConformity.cpp

The `MeshConformity` module determines whether two meshes share a conformal interface. The implementation is intentionally generic: the caller provides two meshes, while the class determines which pair of boundaries form the common interface.

4.1 checkConformity: Public Interface

The method tests every combination of boundaries until a conformal interface is found.

```

1  bool MeshConformity::checkConformity(
2      const Mesh& mesh1,
3      const Mesh& mesh2
4  ) {
5      std::vector<Boundary> all_boundaries = {
6          Boundary::Top,
7          Boundary::Bottom,
8          Boundary::Left,
9          Boundary::Right
10     };
11
12     for (Boundary b1 : all_boundaries) {
13         for (Boundary b2 : all_boundaries) {
14
15             if (checkInterface(mesh1, b1, mesh2, b2)) {
16                 return true;
17             }
18         }
19     }
20 }

```

```

21     return false;
22 }

```

The method therefore avoids requiring the user to specify the interface location explicitly. It returns `true` when any boundary pair matches and `false` otherwise.

4.2 checkInterface: Interface Verification

The `checkInterface` method performs the actual coordinate-based comparison.

```

1  bool MeshConformity::checkInterface(
2      const Mesh& mesh1,
3      Boundary b1,
4      const Mesh& mesh2,
5      Boundary b2
6  ) {
7      auto edge1 = getBoundary(mesh1, b1);
8      auto edge2 = getBoundary(mesh2, b2);
9
10     if (edge1.size() != edge2.size()) {
11         return false;
12     }
13
14     for (const Point* p1 : edge1) {
15
16         auto isMatch = [p1](const Point* p2) {
17             return std::abs(p1->x - p2->x) < 1e-6 &&
18                    std::abs(p1->y - p2->y) < 1e-6;
19         };
20
21         if (std::find_if(
22             edge2.begin(),
23             edge2.end(),
24             isMatch
25         ) == edge2.end()) {
26             return false;
27         }
28     }
29
30     return true;
31 }

```

The algorithm first compares the number of points on the two candidate boundaries. It then uses `std::find_if` to locate a coordinate match for every point on the first boundary. A point without a corresponding point on the second boundary indicates a non-conformal interface.

5 MeshQuality.cpp

The `MeshQuality` module provides an automated geometric validation stage for the computational grid. It is intended to prevent invalid cells, such as inverted or excessively distorted elements, from being passed to the numerical solver.

5.1 Mathematical Diagnostics

For each quadrilateral cell with nodes p_0 , p_1 , p_2 , and p_3 , the implementation evaluates three geometric metrics.

5.1.1 Signed Area

The signed area is evaluated using the shoelace formula:

$$A = \frac{1}{2} \sum_{i=0}^3 (x_i y_{i+1} - x_{i+1} y_i),$$

with cyclic indexing. A non-positive value indicates an invalid orientation for the purposes of the current quality test.

5.1.2 Scaled Corner Jacobian

At each cell corner, a normalized cross product is used to measure local geometric orientation and distortion. For edge vectors $\vec{a}$ and $\vec{b}$,

$$J = \frac{a_x b_y - a_y b_x}{\sqrt{a_x^2 + a_y^2} \sqrt{b_x^2 + b_y^2}}.$$

A non-positive minimum corner Jacobian flags an inverted or degenerate cell.

5.1.3 Aspect Ratio

The aspect ratio measures grid stretching by comparing the characteristic dimensions of the cell. Cells whose aspect ratio exceeds the user-defined limit are classified as low quality.

5.2 Core Diagnostic Implementations

The following functions implement the signed-area and corner-Jacobian calculations.

```

1 double MeshQuality::computeSignedArea(
2     const Point& p0,
3     const Point& p1,
4     const Point& p2,
5     const Point& p3
6 ) {
7     double area = 0.0;
8
9     area += p0.x * p1.y - p1.x * p0.y;
10    area += p1.x * p2.y - p2.x * p1.y;
11    area += p2.x * p3.y - p3.x * p2.y;
12    area += p3.x * p0.y - p0.x * p3.y;
13
14    return 0.5 * area;
15 }
16
17 double MeshQuality::computeCornerJacobian(
18     const Point& prev,
```

```

19     const Point& curr,
20     const Point& next
21 ) {
22     double ax = curr.x - prev.x;
23     double ay = curr.y - prev.y;
24     double bx = next.x - curr.x;
25     double by = next.y - curr.y;
26
27     double cross = ax * by - ay * bx;
28     double lenA = sqrt(ax * ax + ay * ay);
29     double lenB = sqrt(bx * bx + by * by);
30     double denom = lenA * lenB;
31
32     if (denom < numeric_limits<double>::epsilon()) {
33         return 0.0;
34     }
35
36     return cross / denom;
37 }

```

5.3 Quality Evaluation Logic

The `evaluate` method applies the diagnostic metrics to every cell in the mesh.

- **Inverted elements:** a cell is flagged when its minimum scaled corner Jacobian is ≤ 0 .
- **Low-quality elements:** a non-inverted cell is flagged when its aspect ratio exceeds the user-defined threshold.

```

1  vector<CellQuality> MeshQuality::evaluate(
2      const Mesh& mesh,
3      double aspectRatioLimit
4  ) {
5      vector<CellQuality> results;
6      results.reserve(mesh.cells.size());
7
8      for (const auto& cell : mesh.cells) {
9
10         const Point& p0 = mesh.nodes[cell.nodeIDs[0]];
11         const Point& p1 = mesh.nodes[cell.nodeIDs[1]];
12         const Point& p2 = mesh.nodes[cell.nodeIDs[2]];
13         const Point& p3 = mesh.nodes[cell.nodeIDs[3]];
14
15         double signedArea =
16             computeSignedArea(p0, p1, p2, p3);
17
18         double j0 =
19             computeCornerJacobian(p3, p0, p1);
20         double j1 =
21             computeCornerJacobian(p0, p1, p2);
22         double j2 =
23             computeCornerJacobian(p1, p2, p3);
24         double j3 =

```

```

25         computeCornerJacobian(p2, p3, p0);
26
27         double minJacobian =
28             min(min(j0, j1), min(j2, j3));
29
30         double aspectRatio =
31             computeAspectRatio(p0, p1, p2, p3);
32
33         CellQuality quality;
34         quality.id = cell.id;
35         quality.signedArea = signedArea;
36         quality.minJacobian = minJacobian;
37         quality.aspectRatio = aspectRatio;
38
39         quality.inverted =
40             (minJacobian <= 0.0);
41
42         quality.lowQuality =
43             (!quality.inverted &&
44              aspectRatio > aspectRatioLimit);
45
46         results.push_back(quality);
47     }
48
49     return results;
50 }

```

6 Automated Testing Suite

The `testConformity.cpp` test program serves as the entry point for automated validation of the mesh-generation pipeline. It exercises geometry creation, Cartesian discretization, coordinate mappings, conformity checks, and mesh quality diagnostics.

6.1 Geometry Initialization

The test begins by defining the points and boundary lines used to construct the base surfaces.

```

1  int main() {
2      Test test;
3
4      // Define Block 1 Geometry
5      Point p0(0, 0.0, 0.0);
6      Point p1(1, 1.0, 0.0);
7      Point p2(2, 1.0, 1.0);
8      Point p3(3, 0.0, 1.0);
9
10     Line l0(0, &p0, &p1);
11     Line l1(1, &p1, &p2);
12     Line l2(2, &p2, &p3);
13     Line l3(3, &p3, &p0);
14
15     Surface block1Surface(0);
16     block1Surface.addBoundary(&l0);

```

```

17     block1Surface.addBoundary(&l1);
18     block1Surface.addBoundary(&l2);
19     block1Surface.addBoundary(&l3);
20
21     // Define Block 2 Geometry
22     Point p4(4, 0.0, 0.0);
23     Point p5(5, 1.0, 0.0);
24     Point p6(6, 1.0, 1.0);
25     Point p7(7, 0.0, 1.0);
26
27     Line l4(4, &p0, &p1);
28     Line l5(5, &p1, &p2);
29     Line l6(6, &p2, &p3);
30     Line l7(7, &p3, &p0);
31
32     Surface block2Surface(1);
33     block2Surface.addBoundary(&l4);
34     block2Surface.addBoundary(&l5);
35     block2Surface.addBoundary(&l6);
36     block2Surface.addBoundary(&l7);

```

6.2 Computational Domain Discretization

Once the surfaces are defined, the framework generates structured Cartesian ξ - η grids. A 10×10 nodal distribution is used in the test.

```

1  // Create xi_eta mesh domains
2  Mesh xi_eta_1, xi_eta_2;
3
4  xi_eta_1.generateCartesian(
5      block1Surface,
6      10,
7      10
8  );
9
10 xi_eta_2.generateCartesian(
11     block2Surface,
12     10,
13     10
14 );

```

6.3 Coordinate Mappings

The computational grids are passed to the `CoordinateMapping` module to obtain the final physical geometry. The test uses a 90° pipe bend and an outlet extension.

```

1  // Apply physical transformations
2  CoordinateMapping::PipeBend(
3      xi_eta_1,
4      acos(-1) / 2, // 90 degree angle
5      5,             // Inner radius
6      10            // Outer radius
7  );

```

```

8
9 CoordinateMapping::outlet(
10     xi_eta_2,
11     5,           // Length
12     acos(-1) / 2, // Anchor angle
13     5,           // Inner radius
14     10           // Outer radius
15 );

```

6.4 Automated Validation

The transformed grids are subjected to a set of programmatic tests before export.

```

1 // Test 1: Conformal interface verification
2 test.expectTrue(
3     MeshConformity::checkConformity(xi_eta_1, xi_eta_2),
4     "Mesh Conformity Check"
5 );
6
7 // Evaluate global mesh quality
8 auto results =
9     MeshQuality::evaluate(xi_eta_1, 5);
10
11 // Test 2: Inverted-cell detection
12 test.expectTrue(
13     !MeshQuality::hasInvertedCells(results),
14     "Inverted Cells Check"
15 );
16
17 // Test 3: Aspect-ratio / low-quality-cell detection
18 test.expectTrue(
19     !MeshQuality::hasLowQualityCells(results),
20     "Low Quality Cells Check"
21 );
22
23 // Print the test summary and return an appropriate exit code
24 test.summary();
25
26 return test.success() ? 0 : 1;
27 }

```

7 Summary

The implementation establishes a complete multi-block mesh-generation and validation workflow. The main program constructs the computational blocks and applies coordinate mappings, while `MeshConformity` verifies the interfaces between adjacent blocks and `MeshQuality` validates the resulting cells. The automated test suite then exercises these components before the final mesh is accepted for downstream use.